Data Cleaning

JOU-YI LEE
UC Santa Cruz, M.S. in NLP
65 Rio Robles E
San Jose, CA 95134
jlee1039@ucsc.edu

Abstract

This project explores sentiment analysis using the Twitter US Airline Sentiment dataset, and this project are focusing on data analysis, text preprocessing, and classification model development. and this project is divided into five parts:

1. **Dataset Analysis:** exploration of the dataset using Pandas, including per-airline sentiment distribution, tweet length analysis, and visualization of the findings through histograms.
2. **Data Cleaning:** Designed and implemented a data preprocessing pipeline, using regular expressions to removing URLs, emojis, and other entities.
3. **Model Training and Evaluation:** Trained a linear SVM classifier using SGD with TF-IDF encoded tweets. A 10-fold cross-validation approach to evaluate data cleaning strategies through an ablation study.
4. **Additional Analysis:** Exploration of the dataset by identifying unique users, their top words using TF-IDF, and the most active users for each airline.

This work demonstrates a systematic approach to sentiment analysis.

Introduction

1 Twitter US Airline Sentiment Dataset

The Twitter US Airline Sentiment Dataset is a dataset in the field of natural language processing (NLP). It consists of 14,640 tweets directed at major US airlines such as United, Delta, Southwest, and Virgin America, collected in February 2015. Each tweet is labeled with one of three sentiment categories: positive, neutral, or negative, with additional annotations for the reasons behind negative sentiments.

The dataset includes multiple features:

- **Tweet Content**: The text of the tweet, which forms the primary input for NLP tasks.
- **Airline Sentiment**: A categorical label indicating the sentiment of the tweet.
- **Negative Reason**: If the sentiment is negative, this field explains the reason (e.g., "Bad Flight").
- **Confidence Scores**: Fields such as *airline_sentiment_confidence* and *negativereason_confidence* provide confidence levels for the sentiment and negative reason annotations.
- **Metadata**: Fields like *airline*, *name*, *tweet_location*, and *tweet_created* provide contextual information about the tweet.

The Twitter US Airline Sentiment contains several columns:

- **tweet_id**: The unique identifier of the tweet.
- **airline_sentiment**: The sentiment label of the tweet (positive, negative, neutral).
- **airline_sentiment_confidence**: The confidence level of the sentiment label.
- **negativereason**: If the sentiment is negative, this field explains the reason (e.g., "Bad Flight").
- **negativereason_confidence**: The confidence level of the negative reason annotation.
- **airline**: The airline's name (e.g., "Virgin America").
- **name**: The name of the user who tweeted.
- **retweet_count**: The number of times the tweet was retweeted.
- **text**: The content of the tweet.

- **tweet_coord**: The geographic coordinates of the tweet.
- **tweet_created**: The timestamp when the tweet was created.
- **tweet_location**: The location of the user who tweeted.
- **user_timezone**: The user's time zone.

This dataset is particularly valuable for tasks like sentiment analysis as it combines real-world textual data with structured labels.

1.1 Tweet Counts per Airline

- **United Airlines**: 3822 tweets, the airline with the highest number of tweets in the dataset.
- **US Airways**: 2913 tweets, ranking second.
- **American Airlines**: 2759 tweets.
- **Southwest Airlines**: 2420 tweets.
- **Delta Airlines**: 2222 tweets.
- **Virgin America**: 504 tweets, the airline with the least tweets in the dataset.

1.2 Airline Sentiment Statistics

- **Mean**: The average number of tweets per sentiment is 4880.
- **Standard Deviation (std)**: The variance in the number of tweets across different sentiments is 3740, indicating significant differences in distribution.
- **Minimum (min)**: The sentiment category with the least tweets has 2363.
- **Maximum (max)**: The sentiment category with the most tweets has 9178.

1.3 Negative Reason Statistics

- **Mean**: Each negative reason appears, on average, 917.8 times.
- **Standard Deviation (std)**: The frequency variance is 840.58, indicating uneven distribution across reasons.
- **Minimum (min)**: The least common reason appears 74 times.
- **Maximum (max)**: The most common reason appears 2910 times.

1.4 Tweet Length Statistics

- **Shortest Tweet Length**: 12 characters, indicating the presence of very short tweets.
- **Longest Tweet Length**: 186 characters, likely close to Twitter's original character limit of 140 or 280 characters at the time.

1.5 Distribution of Tweet Lengths

The histogram in Figure 1 shows the distribution of tweet lengths:

- **X-axis**: Tweet length in characters.
- **Y-axis**: Frequency of tweets corresponding to each length.
- **Observation**: Most tweets cluster around 140 characters, likely due to Twitter's character limit. Short tweets (fewer than 60 characters) occur less frequently, indicating users prefer expressing complete thoughts. Overall, the distribution exhibits right-skewness, with most tweets below 140 characters. Adjusting the character limit could provide more insights into user preferences.

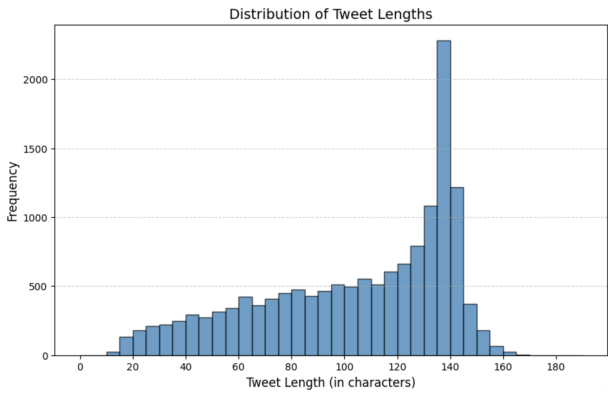

Figure 1: Distribution of Tweet Lengths

1.6 Tweet Sentiment Distribution per Airline

- **X-axis**: Sentiment categories: Positive (green), Neutral (blue), Negative (red).
- **Y-axis**: Frequency of tweets for each sentiment.

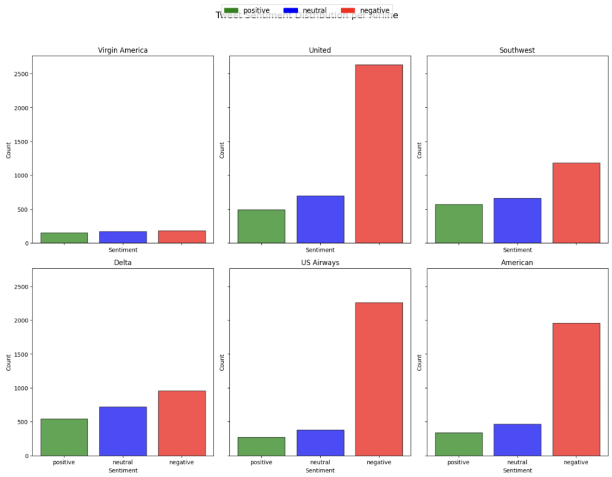

Figure 2: Tweet Sentiment Distribution per Airline

1.7 Tokenization

Tokenization is the process of breaking down text into smaller units (tokens) such as words, punctuation marks, or special entities like URLs and emails. Here is an explanation of the eight tokenization rules implemented in the `custom_tokenizer` function:

- **URLs**: Replace any URL with the token `"URL"`.
  Regex: `r"http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@&+]|[!* \(\),]|(?:%[0-9a-fA-F][0-9a-fA-F]))+"`.
- **Emails**: Replace email addresses with the token `"EMAIL"`.
  Regex: `r"\b\w+@\w+\.\w+\b"`.
- **Numbers and Letters Separation**: Insert a space between numbers and letters.
  Regex: `r"(\d+)([a-zA-Z]+)"` and `r"([a-zA-Z]+)(\d+)"`.
- **Punctuation**: Add spaces around punctuation marks.
  Regex: `r"([.,!?;:])"`.
- **Contractions**: Handle contractions like `"don't"` into `"do n't"`.
  Regex: `r"\b'\b"`.
- **Extra Spaces**: Replace multiple spaces with a single space.
  Regex: `r"\s+"`.
- **User Mentions**: Remove the `@` symbol but retain usernames.
  Regex: `r"@\(\w+)"`.
- **Hashtags**: Remove the # symbol but retain hashtags.
  Regex: `r"#\(\w+)"`.

1.8 Comparison: NLTK's Tokenizer vs. Custom Tokenizer

Custom Tokenizer: Uses predefined rules for specific patterns, allowing for customization. For example, it removes symbols like `@` or # while preserving the main content (e.g., usernames and hashtags).

NLTK Tokenizer: Relies on generalized rules, maintaining the context of the text. For instance, it treats `"don't"` as one token and retains # in hashtags.

1.8.1 Examples and Differences

Example 1:

- **Original Text**: `@SouthwestAir you're my early frontrunner for best airline! #oscars2016`.
- **Custom Tokenizer Output**: `['SouthwestAir', 'you', "'", 're', 'my', 'early', 'frontrunner', 'for', 'best', 'airline', '', 'oscars', '2016']`.
- **NLTK Tokenizer Output**: `['', 'SouthwestAir', 'you', "'re", 'my', 'early', 'frontrunner', 'for', 'best', 'airline', '', '#', 'oscars2016']`.
- **Differences**: The custom tokenizer removes `@` and #, splitting `oscars2016` into `oscars` and `2016`, while NLTK retains `@`, #, and considers `oscars2016` as a single token.

Example 2:

- **Original Text**: `@USAirways how is it that my flt to EWR was Cancelled Flightled yet flts to NYC from USAirways are still flying?`.
- **Custom Tokenizer Output**: `['USAirways', 'how', 'is', 'it', 'that', 'my', 'flt', 'to', 'EWR', 'was', 'Cancelled', 'Flightled', 'yet', 'flts', 'to', 'NYC', 'from', 'USAirways', 'are', 'still', 'flying', '?']`.

- **NLTK Tokenizer Output**: ['@', 'USAirways', 'how', 'is', 'it', 'that', 'my', 'flt', 'to', 'EWR', 'was', 'Cancelled', 'Flightled', 'yet', 'flts', 'to', 'NYC', 'from', 'USAirways', 'are', 'still', 'flying', '?'].

- **Differences**: The custom tokenizer removes the @ symbol, while NLTK retains it. Otherwise, both tokenizers produce similar results.

Example 3:

- **Original Text**: @JetBlue what is going on with your BDL to DCA flights yesterday and today?! Why is every single one getting delayed?.

- **Custom Tokenizer Output**: ['JetBlue', 'what', 'is', 'going', 'on', 'with', 'your', 'BDL', 'to', 'DCA', 'flights', 'yesterday', 'and', 'today', '?', '!', 'Why', 'is', 'every', 'single', 'one', 'getting', 'delayed', '?'].

- **NLTK Tokenizer Output**: ['@', 'JetBlue', 'what', 'is', 'going', 'on', 'with', 'your', 'BDL', 'to', 'DCA', 'flights', 'yesterday', 'and', 'today', '?', '!', 'Why', 'is', 'every', 'single', 'one', 'getting', 'delayed', '?'].

- **Differences**: The custom tokenizer removes the @ symbol, while NLTK retains it. Punctuation handling remains consistent across both tokenizers.

Example 4:

- **Original Text**: @JetBlue do they have to depart from Washington, D.C.??.

- **Custom Tokenizer Output**: ['JetBlue', 'do', 'they', 'have', 'to', 'depart', 'from', 'Washington', ',', 'D', '.', 'C', '.', '?', '?'].

- **NLTK Tokenizer Output**: ['@', 'JetBlue', 'do', 'they', 'have', 'to', 'depart', 'from', 'Washington', ',', 'D.C.', '?', '?'].

- **Differences**: The custom tokenizer splits D.C. into separate tokens D . C ., while NLTK treats D.C. as a single token. Additionally, the custom tokenizer removes the @ symbol.

Example 5:

- **Original Text**: @JetBlue I can probably find some of them. Are the ticket #s on there?.

- **Custom Tokenizer Output**: ['JetBlue', 'I', 'can', 'probably', 'find', 'some', 'of', 'them', '.', 'Are', 'the', 'ticket', 's', 'on', 'there', '?'].

- **NLTK Tokenizer Output**: ['@', 'JetBlue', 'I', 'can', 'probably', 'find', 'some', 'of', 'them', '.', 'Are', 'the', 'ticket', '#', 's', 'on', 'there', '?'].

- **Differences**: The custom tokenizer removes the @ and # symbols and splits ticket #s into ticket s. NLTK retains both symbols and considers #s as two separate tokens.

Conclusion: The custom tokenizer provides more fine-grained control by implementing specific rules for processing text, such as removing symbols (@, #), handling contractions, and splitting abbreviations. In contrast, NLTK's tokenizer is more generalized, maintaining context and preserving symbols as part of the text. This makes the custom tokenizer better suited for tasks requiring strict preprocessing, while NLTK's tokenizer is advantageous for tasks relying on the original text structure.

2 SVM Sentiment Classification Model Training

We implemented 14 data cleaning methods to preprocess the dataset for sentiment classification using a Support Vector Machine (SVM). This process effectively removed noise, standardized the text, and ensured no duplicate or empty rows, providing a high-quality dataset for model training and testing. The cleaned data better reflects user sentiments and is suitable for SVM-based sentiment classification.

2.1 Data Cleaning Methods

Method	Description
Mentions	Removes `@username`
URLs	Removes URLs like starting with `http://`
Email Addresses	Removes email like `user@example.com`.
Currency Values	Removes currency values `$19.90` or `$100`.
Emojis	Removes all emojis using Unicode standards.
HTML	Removes HTML encoded characters such as `<`.
Normalize Dates	Removes or normalizes time and date formats like `2/24 2:10pm` or `6/30`.
Punctuation	Removes all punctuation marks such as `!`, `?`, and `.`.
Chinese Characters	Removes all Chinese characters.
Expand Contractions	Expands contractions into their full forms.
Repeated Characters	Reduces consecutive repeating characters to a single occurrence.
Too Short or Long Words	Removes words shorter than 2 characters or longer than 15 characters.
Lemmatization	Converts words to their base forms using WordNet Lemmatizer.
Extra Whitespace	Removes all extra spaces.

Table 1: Data Cleaning Methods and Descriptions

2.2 Final Steps

After applying the above cleaning methods:

- Duplicate rows with identical cleaned tweets and sentiment labels were removed.
- Rows with empty tweets after cleaning were also removed.

2.3 Summary of the Cleaned Dataset

The cleaned dataset has the following properties:

- **Total Rows**: The dataset contains 14,254 rows after cleaning.
- **Unique Airlines**: The dataset includes tweets from 6 unique airlines.
- **Sentiment Distribution**:
  - **Negative**: 9,078 tweets
  - **Neutral**: 2,959 tweets
  - **Positive**: 2,217 tweets
- **Empty Texts**: There are 0 empty texts after cleaning.
- **Duplicate Rows**: There are 0 duplicate rows in the cleaned dataset.

This cleaning process ensures a high-quality dataset suitable for sentiment classification using SVM.

3 SVM Model and TF-IDF (Term Frequency-Inverse Document Frequency)

3.1 Data Splitting

To ensure balanced class distributions in the training and testing sets, I used `stratify` during the data splitting process. The dataset was divided with 90% as the training set and 10% as the testing set. The class distributions are as follows:

- **Training Set:**
  - Negative: 8,170 tweets
  - Neutral: 2,663 tweets
  - Positive: 1,995 tweets
- **Testing Set:**
  - Negative: 908 tweets
  - Neutral: 296 tweets
  - Positive: 222 tweets

The results show that the split is well-balanced, ensuring there are no issues with data imbalance.

3.2 Feature Engineering

In this study, I utilized TF-IDF (Term Frequency-Inverse Document Frequency) to vectorize the text data, extracting up to 5000 feature terms. This step transforms textual data into numerical format, making it suitable for use with SVM models.

3.3 Support Vector Machine Model (SGDClassifier)

I employed the SGDClassifier, a model based on Support Vector Machines (SVM), with the hinge loss function. The hinge loss function is a key element, suitable for binary and multi-class classification problems.

3.3.1 Hyperparameter Setup

The initial hyperparameter configuration for SGDClassifier is as follows:

- Regularization: L2
- Learning Rate: $\alpha = 1e - 4$
- Maximum Iterations: 100

3.3.2 Cross-Validation

To evaluate the model's performance on the training set, I performed 10-fold cross-validation. The average cross-validation accuracy achieved was 80.09%.

3.3.3 Hinge Loss Function

The hinge loss function aims to maximize the margin between data points and the classification hyperplane. This characteristic improves the model's generalization ability, reducing the likelihood of errors when handling unseen data. The formula for the hinge loss is:

$$\text{Hinge Loss} = \max(0, 1 - y \cdot f(x))$$

3.3.4 Testing Set Performance

In the initial model setup, the accuracy on the testing set was 81.14%, with a weighted average F1 score of 80%. This indicates that the overall classification performance was good.

3.4 Classification Performance Summary

- **Negative Class:** This class performed best, with a precision of 84%, recall of 94%, and F1 score of 89%.
- **Neutral Class:** The recall was only 51%, leading to a higher likelihood of misclassification into other classes.
- **Positive Class:** The performance fell between the negative and neutral classes, with a precision of 81%, recall of 69%, and F1 score of 75%.

3.5 Confusion Matrix Analysis

- **Negative Class:** Only 13 tweets were misclassified as positive, indicating that the features for this class are highly distinguishable.
- **Neutral Class:** 120 tweets were misclassified as negative, and 24 tweets were misclassified as positive. This suggests significant feature overlap between the neutral class and others, making classification challenging.
- **Positive Class:** 40 tweets were misclassified as negative, and 28 as neutral, indicating moderate classification performance for this class.

Class	Precision	Recall	F1 Score
Negative	84%	94%	89%
Neutral	68%	51%	58%
Positive	81%	69%	75%
Weighted Avg	-	-	80%

Table 2: Classification Report

Actual \Predicted	Negative	Neutral	Positive
Negative	851	44	13
Neutral	120	152	24
Positive	40	28	154

Table 3: Confusion Matrix

3.6 Optuna